

SİSTEM PROGRAMLAMA - LABORATUVAR 7 RAPORU

Ad: Bartu

Soyad: Şanlı

Numara: 23220030065

Açıklayıcı Video Link:

<https://drive.google.com/file/d/175u2siEdo-I2CuAEJGusc2aNx6xCdtG7/view?usp=sharing>

Konu: IPC (Süreçler Arası İletişim) - Paylaşılan Bellek ve Semaforlar ile Üretici-Tüketici Problemi

1. Giriş

Bu laboratuvar çalışmasının amacı, Unix/Linux işletim sistemlerinde süreçler arası iletişim (IPC) mekanizmalarından **Shared Memory (Paylaşılan Bellek)** ve **Semaphores (Semaforlar)** yapılarının incelenmesidir. Çalışma kapsamında, işletim sistemleri teorisinin temel problemlerinden biri olan "Üretici-Tüketici (Producer-Consumer)" problemi ele alınmıştır. Amaç, ortak bir bellek alanına erişen iki farklı sürecin, veri bütünlüğünü bozmadan ve birbirlerini bloklamadan senkronize bir şekilde çalışmasını sağlamaktır.

2. Sistem Tasarımı ve Kod Analizi

Proje; ortak tanımlamalar, yönetici (parent), üretici (producer) ve tüketici (consumer) olmak üzere dört ana modülden oluşmaktadır.

2.1. Ortak Yapıların Tanımlanması (common.h)

Bu başlık dosyası, üretici ve tüketici süreçlerinin ortaklaşa kullanacağı veri yapılarını ve sabitleri barındırır.

- **Struct MEMORY:** Paylaşılan belleğin yapısını tanımlar. 6 slotluk bir buffer (Dairesel Kuyruk/Circular Buffer) ve kuyruğun baş/son indekslerini tutan head ve tail değişkenlerini içerir.
- **Semafor İşlemleri:** AVAIL_SLOTS (Boş yer sayısı) ve TO_CONSUME (Dolu yer sayısı) olmak üzere iki semafor indeksi tanımlanmıştır.

```

GNU nano 7.2                                common.h *
/* common.h */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>

#define MSGNUMBER 10                        /* Tüketicideki farklı mesaj sayısı */
#define SLOT_LEN 50                        /* Bir mesajdaki max karakter sayısı */
#define N_SLOTS 6                          /* Kuyruktaki max boş slot sayısı (Buffer bo>
#define MAXMSGS 25                         /* Üretilen ya da tüketilen max mesaj sayısı>

/* Paylaşılmış bellek yapısı */
struct MEMORY {
    char buffer[N_SLOTS][SLOT_LEN]; /* Mesajlar için array (Dairesel Kuyru>
    int head, tail;                  /* Kuyruğun başı ve sonu için indeksle>
};

/* Semafor operasyonları için struct tanımları */
struct sembuf acquire = {0, -1, SEM_UNDO}; /* Kaynak al (P işlemi / Wait) >
struct sembuf release = {0, 1, SEM_UNDO}; /* Kaynak bırak (V işlemi / Sig>

/* Semafor indeksleri */
enum {AVAIL_SLOTS, TO_CONSUME};
/* AVAIL_SLOTS (0): Boş slot sayısını tutar.
   TO_CONSUME (1): Dolu (tüketilecek) slot sayısını tutar.
*/

```

(Sonradan #include <sys/wait.h> satırı da eklenmiştir.)

2.2. Yönetici Süreç (prod_cons_parent.c)

Bu modül, sistemin başlatıcısı ve temizleyicisidir.

- **Başlatma (Initialization):** shmget ile paylaşılan bellek alanını ve semget ile semafor kümesini oluşturur. Bellek alanını sıfırlar.
- **Semafor Ayarları:** Başlangıçta kuyruk boş olduğu için AVAIL_SLOTS değerini tampon boyutu olan 6'ya, TO_CONSUME değerini ise 0'a eşitler.
- **Süreç Yönetimi:** fork() ve exec() fonksiyonlarını kullanarak Üretici ve Tüketici programlarını çocuk süreç (child process) olarak başlatır.
- **Temizlik:** wait() fonksiyonu ile çocukların bitmesini bekler ve ardından IPC_RMID komutuyla bellek ve semaforları sistemden siler. Bu adım, sistemde "zombie" kaynak kalmaması için kritiktir.

```
/* prod_cons_parent.c */
#include "common.h"

int main(int argc, char *argv[]) {
    static struct MEMORY sharmemory;
    static ushort init_val[2] = {N_SLOTS, 0}; /* AVAIL_SLOTS = 6 (Başlangıç>
                                                TO_CONSUME = 0 (Başlangıçta>
                                                */

    int semid, shmid, child;
    char *shmptr;
    int prod_id, cons_id, parent_id;
    union semun {
        int val;
        struct semid_ds *buf;
        ushort *array;
    } arguments;

    printf("Parent process basliyor...\n");

    if (argc != 2) {
        printf("Kullanım: %s mesaj_sayisi\n", argv[0]);
        exit(1);
    }

    /* 1. Paylaşılmış Belleği Yarat ve Bağla */
    parent_id = getpid();
    shmid = shmget(parent_id, sizeof(sharmemory), IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget problem"); exit(1); }

    shmptr = shmat(shmid, 0, 0);
    if (shmptr == (char *) -1) { perror("shmat problem"); exit(1); }

    /* Belleği sıfırla */
    sharmemory.head = sharmemory.tail = 0;
    memcpy(shmptr, (char *)&sharmemory, sizeof(sharmemory));
    printf("Paylaşılmis Bellek hazir...\n");
}
```

```

/* 2. Semaforları Yarat ve İlklendir */
semid = semget(parent_id, 2, IPC_CREAT | 0666);
if (semid == -1) { perror("semget problem"); exit(1); }

arguments.array = init_val;
if (semctl(semid, 0, SETALL, arguments) == -1) { perror("semctl problem"); }
printf("Semaphorlar hazır...\n");

/* 3. Child Prosesleri Başlat */
prod_id = fork();
if (prod_id == 0) execl("./producer", "producer", argv[1], NULL);

cons_id = fork();
if (cons_id == 0) execl("./consumer", "consumer", argv[1], NULL);

/* 4. Bekle ve Temizle */
printf("Parent process çocukları bekliyor...\n");
wait(NULL);
wait(NULL);

/* Belleği ve Semaforu Sil */
shmdt(shmptr);
shmctl(shmid, IPC_RMID, 0);
semctl(semid, 0, IPC_RMID, 0);

printf("\nTüm processler sonlandı ve temizlik yapıldı.\n");
exit(0);
}

```

,2.3. Üretici Süreç (producer.c)

Üretici, önceden tanımlı mesajları sırasıyla paylaşılan belleğe yazar.

- **Kritik Bölgeye Giriş (Wait):** Veriyi yazmadan önce AVAIL_SLOTS semaforunu kontrol eder. Eğer boş yer yoksa (değer 0 ise), işletim sistemi tarafından bloklanır ve yer açılmasını bekler.
- **Veri Yazma:** Boş yer olduğunda semafor değerini azaltır, veriyi buffer[tail] konumuna yazar ve tail indeksini günceller.
- **Kritik Bölgeden Çıkış (Signal):** Yazma işlemi bittiğinde TO_CONSUME semaforunu artırarak Tüketiciye "Okunacak yeni veri var" sinyali gönderir.

```
/* producer.c */
#include "common.h"

int main(int argc, char *argv[]) {
    static char *messages[MSGNUMBER] = {
        "First message", "Second message", "Third message", "Fourth message",
        "Fifth message", "Sixth message", "Seventh message", "Eighth message",
        "Ninth message", "Tenth message"
    };

    static char local[SLOT_LEN];
    static struct MEMORY *memptr;
    char *shmptr;
    int myparid, semid, shmid, i, j, sleeptime, maxmsg, curmsg;

    myparid = getppid(); /* Parent ID'yi al (Key olarak kullanılacak) */
    maxmsg = atoi(argv[1]);
    printf("URETICI basladi... (Uretilecek: %d)\n", maxmsg);
    srand((unsigned)getpid());

    /* Paylaşılan belleğe eriş */
    shmid = shmget(myparid, 0, 0);
    if (shmid < 0) { perror("Uretici: shmget hata"); exit(1); }

    shmptr = shmat(shmid, 0, 0);
    if (shmptr == (char *) -1) { perror("Uretici: shmat hata"); exit(1); }

    memptr = (struct MEMORY *) shmptr;

    /* Semaforlara eriş */
    semid = semget(myparid, 2, 0);
    if (semid == -1) { perror("Uretici: semget hata"); exit(1); }

    /* Ana Döngü */
    for (curmsg = 0; curmsg < maxmsg; curmsg++) {
        memset(local, '\0', sizeof(local));
        sleeptime = rand() % 6; /* 0-5 sn bekle */
    }
}
```



```

sleeptime = rand() % 6; /* 0-5 sn bekle */
sleep(sleeptime);

i = curmsg % MSGNUMBER;
strcpy(local, messages[i]); /* Mesajı hazırla */

printf("Uretici hazirladi: %s\n", local);

/* KRİTİK BÖLÜM BAŞLANGICI */
/* Boş slot var mı diye kontrol et (Wait) */
acquire.sem_num = AVAIL_SLOTS;
if (semop(semid, &acquire, 1) == -1) { perror("semop acquire hata")>

/* Mesajı belleğe yaz */
strcpy(memptr->buffer[memptr->tail], local);
memptr->tail = (memptr->tail + 1) % N_SLOTS; /* Dairesel kuyruk man>

/* Tüketiciye tüketilecek veri olduğunu haber ver (Signal) */
release.sem_num = TO_CONSUME;
if (semop(semid, &release, 1) == -1) { perror("semop release hata")>
/* KRİTİK BÖLÜM BİTİŞİ */
}

printf("Uretici sonlandi.\n");
exit(0);
}

```

2.4. Tüketici Süreç (consumer.c)

Tüketici, paylaşılan bellekten veri okuyan süreçtir.

- **Kritik Bölgeye Giriş (Wait):** TO_CONSUME semaforunu kontrol eder. Eğer okunacak veri yoksa (değer 0 ise) üreticinin veri yazmasını bekler.
- **Veri Okuma:** Veri varsa semaforu azaltır, buffer[head] konumundaki veriyi yerel belleğine kopyalar ve head indeksini kaydırır.
- **Kritik Bölgeden Çıkış (Signal):** Okuma işlemi bittiğinde AVAIL_SLOTS semaforunu artırarak Üreticiye "Bufferda yer açıldı" sinyali gönderir.

```

GNU nano 7.2                                     consumer.c *
/* consumer.c */
#include "common.h"

/* DENEY ADIM 2 İÇİN BU DEĞERİ 10 YAPACAĞIZ */
#define MAXSLEEPTIME 5

int main(int argc, char *argv[]) {
    static char local[SLOT_LEN];
    static struct MEMORY *memptr;
    char *shmptr;
    int myparid, semid, shmid, j, sleeptime, maxmsg, curmsg;

    myparid = getppid();
    maxmsg = atoi(argv[1]);
    printf("TUKETICI basladi...\n");
    srand((unsigned)getpid());

    /* Paylaşılan belleğe eriş */
    shmid = shmget(myparid, 0, 0);
    if (shmid < 0) { perror("Tuketici: shmget hata"); exit(1); }

    shmptr = shmat(shmid, 0, 0);
    if (shmptr == (char *) -1) { perror("Tuketici: shmat hata"); exit(1); }

    memptr = (struct MEMORY *) shmptr;

    /* Semaforlara eriş */
    semid = semget(myparid, 2, 0);
    if (semid == -1) { perror("Tuketici: semget hata"); exit(1); }

    /* Ana Döngü */
    for (curmsg = 0; curmsg < maxmsg; curmsg++) {
        memset(local, '\0', sizeof(local));
        sleeptime = rand() % MAXSLEEPTIME;
        sleep(sleeptime);

```

```

        /* KRİTİK BÖLÜM BAŞLANGICI */
        /* Tüketilecek veri var mı bekle (Wait) */
        acquire.sem_num = TO_CONSUME;
        if (semop(semid, &acquire, 1) == -1) { perror("semop acquire hata"); }

        /* Veriyi al */
        strcpy(local, memptr->buffer[memptr->head]);
        memptr->head = (memptr->head + 1) % N_SLOTS;

        /* Üreticiye yer açıldığını haber ver (Signal) */
        release.sem_num = AVAIL_SLOTS;
        if (semop(semid, &release, 1) == -1) { perror("semop release hata"); }
        /* KRİTİK BÖLÜM BİTİŞİ */

        printf(".....Tuketici mesajı aldı: %s\n", local);
    }

    printf("Tuketici sonlandi.\n");
    exit(0);
}

```

3. Deney Sonuçları ve Gözlemler

Kodlar derlendikten sonra üç farklı senaryo üzerinde test edilmiştir.

Durum 1: Normal Çalışma (Senkronizasyon Mevcut)

Bu senaryoda üretici ve tüketici rastgele bekleme süreleriyle çalıştırılmıştır.

- **Gözlem:** Üreticinin yazdığı her mesaj, sırası bozulmadan tüketici tarafından okunmuştur. semop fonksiyonları sayesinde bir süreç yazarken diğeri belleğin o bölgesine müdahale etmemiştir.
- **Sonuç:** Veri bütünlüğü tam olarak sağlanmıştır.

```
bartu_sanli@bartu-computer:~/lab7$ ./parent 10
Parent process basliyor...
Paylasilmis Bellek hazir...
Semaphorlar hazir...
Parent process cocuklari bekliyor...
URETICI basladi... (Uretilecek: 10)
TUKETICI basladi...
Uretici hazirladi: First message
.....Tuketici mesaji aldi: First message
Uretici hazirladi: Second message
Uretici hazirladi: Third message
.....Tuketici mesaji aldi: Second message
Uretici hazirladi: Fourth message
Uretici hazirladi: Fifth message
Uretici hazirladi: Sixth message
.....Tuketici mesaji aldi: Third message
Uretici hazirladi: Seventh message
.....Tuketici mesaji aldi: Fourth message
Uretici hazirladi: Eighth message
.....Tuketici mesaji aldi: Fifth message
.....Tuketici mesaji aldi: Sixth message
.....Tuketici mesaji aldi: Seventh message
Uretici hazirladi: Ninth message
.....Tuketici mesaji aldi: Eighth message
.....Tuketici mesaji aldi: Ninth message
Uretici hazirladi: Tenth message
Uretici sonlandi.
```

Durum 2: Yavaş Tüketici (Slow Consumer / Bounded Buffer)

Tüketici sürecine uzun bir bekleme süresi (sleep) eklenerek yavaşlatılmıştır.

- **Gözlem:** Üretici çok hızlı bir şekilde buffer'daki 6 slotu doldurmuştur. 7. mesajı yazmak istediğinde, boş yer olmadığı için (AVAIL_SLOTS = 0) sistem tarafından durdurulmuş (wait state) ve tüketicinin veri okuyup yer açmasını beklemiştir.


```
/* DENEY ADIM 2 İÇİN BU DEĞERİ 10 YAPACAĞIZ */  
#define MAXSLEEPTIME 10
```

- **Sonuç:** Sınırlı tampon (Bounded Buffer) problemi semaforlar sayesinde taşma (overflow) olmadan çözülmüştür.

```
bartu_sanli@bartu-computer:~/lab7$ ./parent 10  
Parent process basliyor...  
Paylasilmis Bellek hazir...  
Semaphorlar hazir...  
Parent process cocuklari bekliyor...  
URETICI basladi... (Uretilecek: 10)  
TUKETICI basladi...  
Uretici hazirladi: First message  
.....Tuketici mesaji aldi: First message  
Uretici hazirladi: Second message  
Uretici hazirladi: Third message  
Uretici hazirladi: Fourth message  
.....Tuketici mesaji aldi: Second message  
Uretici hazirladi: Fifth message  
.....Tuketici mesaji aldi: Third message  
Uretici hazirladi: Sixth message  
Uretici hazirladi: Seventh message  
.....Tuketici mesaji aldi: Fourth message  
Uretici hazirladi: Eighth message  
Uretici hazirladi: Ninth message  
.....Tuketici mesaji aldi: Fifth message  
Uretici hazirladi: Tenth message  
Uretici sonlandi.
```

Durum 3: Yariş Durumu (Race Condition)

Semafor kontrolleri (acquire ve release blokları) koddan çıkarılarak sistem korumasız bırakılmıştır.

- **Gözlem:** Senkronizasyon kalktığı için üretici, tüketici okumadan sürekli verilerin üzerine yazmıştır. Tüketici ise veri yazılmamış olsa bile eski veya bozuk verileri mükerrer şekilde okumuştur.

```
/* Semaforlara eriş */  
// semid = semget(myparid, 2, 0);  
// if (semid == -1) { perror("Uretici: semget hata"); exit(1); }
```

```

        /* KRİTİK BÖLÜM BAŞLANGICI */
        /* Boş slot var mı diye kontrol et (Wait) */
//      acquire.sem_num = AVAIL_SLOTS;
//      if (semop(semid, &acquire, 1) == -1) { perror("semop acquire hata"); }

        /* Mesajı belleğe yaz */
        strcpy(memptr->buffer[memptr->tail], local);
        memptr->tail = (memptr->tail + 1) % N_SLOTS; /* Dairesel kuyruk mantığı */

        /* Tüketiciye tüketilecek veri olduğunu haber ver (Signal) */
//      release.sem_num = TO_CONSUME;
//      if (semop(semid, &release, 1) == -1) { perror("semop release hata"); }
        /* KRİTİK BÖLÜM BİTİŞİ */
    }

/* Semaforlara eriş */
//      semid = semget(myparid, 2, 0);
//      if (semid == -1) { perror("Tuketici: semget hata"); exit(1); }

/* Ana Döngü */
for (curmsg = 0; curmsg < maxmsg; curmsg++) {
    memset(local, '\0', sizeof(local));
    sleeptime = rand() % MAXSLEEPTIME;
    sleep(sleeptime);

    /* KRİTİK BÖLÜM BAŞLANGICI */
    /* Tüketilecek veri var mı bekle (Wait) */
//      acquire.sem_num = TO_CONSUME;
//      if (semop(semid, &acquire, 1) == -1) { perror("semop acquire hata"); }

    /* Veriyi al */
    strcpy(local, memptr->buffer[memptr->head]);
    memptr->head = (memptr->head + 1) % N_SLOTS;

    /* Üreticiye yer açıldığını haber ver (Signal) */
//      release.sem_num = AVAIL_SLOTS;
//      if (semop(semid, &release, 1) == -1) { perror("semop release hata"); }
    /* KRİTİK BÖLÜM BİTİŞİ */

    printf(".....Tuketici mesaji aldi: %s\n", local);
}

```

- **Sonuç:** Çıktılarda veri kaybı ve sıralama hataları görülmüştür. Bu durum, paylaşılan belleğin semaforlar olmadan kullanılmasının "Yarış Durumu"na (Race Condition) yol açtığını kanıtlamıştır.

```
bartu_sanli@bartu-computer:~/lab7$ ./parent 10
Parent process basliyor...
Paylasilmis Bellek hazir...
Semaphorlar hazir...
Parent process cocuklari bekliyor...
URETICI basladi... (Uretilecek: 10)
TUKETICI basladi...
.....Tuketici mesaji aldi:
Uretici hazirladi: First message
Uretici hazirladi: Second message
.....Tuketici mesaji aldi:
Uretici hazirladi: Third message
.....Tuketici mesaji aldi:
Uretici hazirladi: Fourth message
Uretici hazirladi: Fifth message
Uretici hazirladi: Sixth message
.....Tuketici mesaji aldi: Fourth message
.....Tuketici mesaji aldi: Fifth message
.....Tuketici mesaji aldi: Sixth message
.....Tuketici mesaji aldi: First message
Uretici hazirladi: Seventh message
.....Tuketici mesaji aldi: Second message
.....Tuketici mesaji aldi: Third message
.....Tuketici mesaji aldi: Fourth message
Tuketici sonlandi.
Uretici hazirladi: Eighth message
Uretici hazirladi: Ninth message
Uretici hazirladi: Tenth message
Uretici sonlandi.

Tum processler sonlandi ve temizlik yapildi.
```

4. Sonuç ve Değerlendirme

Bu laboratuvar çalışması ile süreçler arası iletişimde en hızlı yöntem olan Paylaşılan Bellek (Shared Memory) mimarisi uygulanmıştır. Ancak bu hızın yanında, süreçlerin aynı bellek bölgesine erişiminin kontrol altına alınması gerektiği görülmüştür.

Semaforlar kullanılarak gerçekleştirilen senkronizasyon sayesinde:

1. Veri kaybı önlenmiştir.
2. Bellek taşması (overflow) veya boş veriyi okuma (underflow) hataları engellenmiştir.
3. Birbirinden bağımsız çalışan süreçlerin uyum içinde çalışması sağlanmıştır.

Sistem Programlama dersi kapsamında edinilen bu tecrübe, çoklu işlem (multiprocessing) ve eşzamanlılık (concurrency) konularının temelini oluşturmaktadır.